

Concurrency & Distributed Systems

Week 2 — Part 3: `sleep()` vs `join()`

Dr. Mohamad Aoude

Lebanese University
Faculty of Engineering

May 11, 2026

A Fundamental Coordination Question

Scenario

Thread A must finish before Thread B continues.

Should we:

- Delay using `sleep()`?
- Coordinate using `join()`?

Clocks measure time.

Synchronization observes state.

Core Distinction

sleep()

- Pauses current thread
- Time-based
- Independent of other threads
- TIMED_WAITING state

join()

- Waits for target thread
- Completion-based
- Observes thread state
- WAITING state

Time vs State

Broken Coordination: Concrete Failure

```
// Shared state
static int counter = 0;

Thread worker = new Thread(() -> {
    for (int i = 0; i < 1_000_000; i++) {
        counter++;
    }
});

worker.start();

// WRONG: guess a delay
Thread.sleep(10); // throws InterruptedException

System.out.println(counter);
```

Why This Fails

- May work on fast machine
- Fails under CPU contention
- Fails during GC pauses
- Fails with OS scheduling variation
- Fails in virtualized/cloud environments

Reality

Timing-based coordination is probabilistic, not deterministic.

Before/After Output (What Students Actually See)

Without `join()` (timing guess)

457821 *(wrong: worker still running)*

With `join()` (completion-based)

1000000 *(correct: worker finished)*

This is not “ordering cosmetics” — it is incorrect results.

Why sleep() Cannot Synchronize

- Does NOT observe other threads
- Does NOT wait for completion
- Does NOT guarantee ordering
- Does NOT establish happens-before
- Sensitive to load and scheduling

Correct Coordination with join()

```
worker.start();  
  
worker.join(); // throws InterruptedException  
  
System.out.println(counter);
```

Interrupt Handling (Production Rule)

If interrupted, restore the interrupt status: `Thread.currentThread().interrupt();`

What join() Actually Does

- Calling thread blocks
- JVM checks if target thread is alive
- If alive → WAITING
- If already finished → returns immediately

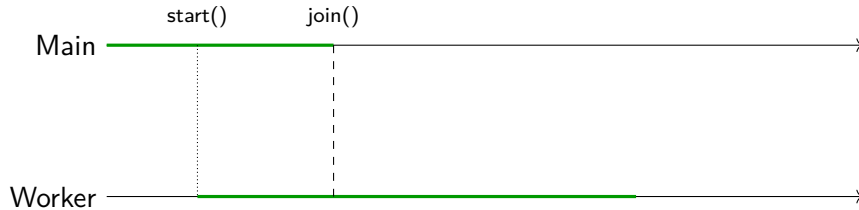
Implementation Insight (Most JVMs)

In many JVM implementations, `join()` is built using `wait()/notifyAll()` on the Thread object's monitor.

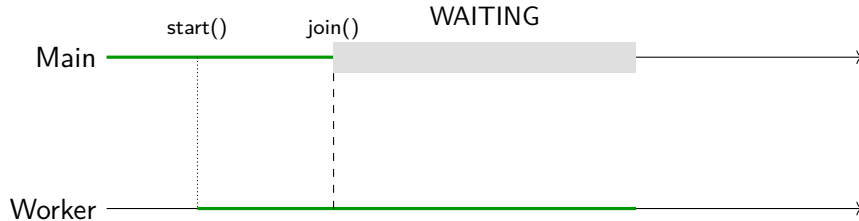
Execution Timeline (State Becomes Visible)



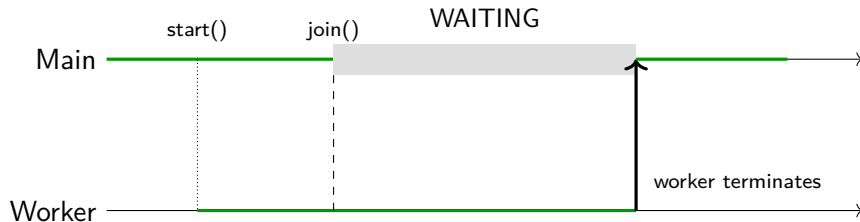
Execution Timeline (State Becomes Visible)



Execution Timeline (State Becomes Visible)



Execution Timeline (State Becomes Visible)



Memory Visibility Guarantee

- `join()` creates a happens-before relationship
- Thread termination happens-before successful return from `join()`

Precise Rule

All writes by worker before termination become visible after `join()` returns.

(Formal rules: JLS §17.4.5)

join(long millis)

```
worker.join(1000);
```

- Waits up to 1 second
- Returns earlier if worker finishes
- If timeout expires, worker may still be alive
- No completion guarantee

Lifecycle Mapping

- `sleep()` → `TIMED_WAITING`
- `join()` → `WAITING`
- Worker terminates → Main becomes `RUNNABLE`

Fork-Join Pattern

```
t1.start();  
t2.start();  
  
t1.join();  
t2.join();
```

- Parallel execution
- Join point enforces completion
- Classic fork-join structure

When Is `sleep()` Acceptable? (With Caveats)

- Rate limiting / backoff (e.g., retry after failure)
- Simulation / demos (explicitly labeled)
- UI animation timing (not correctness-critical)
- Testing support (with clear disclaimers)

Rule

Never use `sleep()` for correctness-critical coordination.

Deep Conceptual Contrast

Time-Based

```
sleep(5000);
```

“I wait 5 seconds.”

State-Based

```
worker.join();
```

“I wait for worker.”

Synchronization is event-driven, not clock-driven.

Reflection Questions

- ❶ If worker takes 10s on a fast machine and 11s on a slow one, does `sleep(10000)` always work? Why?
- ❷ Would `sleep(60000)` be safe in production?
- ❸ What exact memory guarantee does `join()` provide?
- ❹ How is `join()` related to `wait()`/`notifyAll()`?
- ❺ Why is event-based coordination fundamentally safer?

Summary

- `sleep()` delays execution
- `join()` coordinates completion
- `join()` guarantees ordering
- `join()` guarantees memory visibility
- `sleep()` is NOT synchronization

One Rule to Remember

Never coordinate threads using time. Always coordinate using state.